EpistemicLLM

Theory of Mind Reasoning via Epistemic Logic and Large Language Models

Yahya Hamid
yahya6751290@gmail.com
ENSEEIHT, Toulouse INP

April 2026

Abstract

Large Language Models (LLMs) exhibit systematic failures in Theory of Mind (ToM) reasoning — the ability to attribute mental states to other agents. This report presents **EpistemicLLM**, a hybrid system that combines formal epistemic logic with LLMs to address this limitation. The system uses an LLM to extract narrative structure from natural language stories, constructs a Kripke model encoding each agent's belief state, and evaluates epistemic queries with mathematical guarantees and full proof traces.

We evaluate EpistemicLLM on the ToMi benchmark (Nematzadeh et al., 2019) against zero-shot and chain-of-thought baselines. Our system achieves **100% accuracy** on 44 benchmark stories, compared to 54.5% for zero-shot and 97.7% for chain-of-thought. The system supports nested epistemic formulas of arbitrary depth, verified against all five axioms of the S5 epistemic logic system.

Contents

1 Introduction **3**
1.1 The Problem: LLMs and Theory of Mind . . . 3
1.2 The DesiRes Research Programme . . . 3
1.3 Contributions . . . 3

2 Mathematical Background **4**
2.1 Propositional Logic . . . 4
2.2 Modal Logic . . . 4
2.3 Kripke Semantics . . . 4
2.4 Epistemic Logic . . . 5
2.5 The S5 Axiom System . . . 5
2.6 Nested Epistemic Formulas . . . 6
2.7 False Beliefs and Theory of Mind . . . 7

3 System Architecture **7**
3.1 Overview . . . 7
3.2 The Kripke Model (`core/kripke.py`) . . . 7
3.3 The Epistemic Reasoner (`core/reasoner.py`) . . . 8
3.4 The Belief Tracker (`core/tracker.py`) . . . 8
3.5 The Story Parser (`nlp/parser.py`) . . . 9
3.5.1 Story Structure Extraction . . . 9
3.5.2 Query Construction . . . 9
3.6 The Formula Parser (`core/formula_parser.py`) . . . 10
3.7 The 3-Way Comparison Engine (`llm/bridge.py`) . . . 10

4 Experimental Evaluation **10**
4.1 The ToMi Benchmark . . . 10
4.2 Results . . . 11
4.3 Error Analysis . . . 11
4.3.1 Baseline Failures (20 errors) . . . 11
4.3.2 CoT Failure (1 error) . . . 11
4.3.3 EpistemicLLM: Zero Failures . . . 11
4.4 Discussion . . . 11

5 Interactive Demo **12**
5.1 First-Order Mode . . . 12
5.2 Advanced Formula Mode . . . 12

6 Future Research Directions **12**
6.1 Automatic Extraction of Second-Order Formulas . . . 12
6.2 Integration with Causal Reasoning . . . 13
6.3 Argumentation Frameworks . . . 13
6.4 Constrained Decoding . . . 13

7 Conclusion **13**

1 Introduction

1.1 The Problem: LLMs and Theory of Mind

Theory of Mind (ToM) is the cognitive ability to attribute mental states — beliefs, desires, intentions, and knowledge — to other agents, and to understand that those mental states may differ from one's own [10]. It is fundamental to human communication, cooperation, and social reasoning.

Consider the classic Sally-Anne false belief task [1]:

Example 1.1 (Sally-Anne)**.** Sally puts a ball in a basket and leaves the room. Anne moves the ball to a box. Sally returns. *Where does Sally think the ball is?*

The correct answer is "in the basket" — Sally did not witness the move, so her belief about the ball's location is outdated. Children below age 4 and certain clinical populations fail this task. Recent work shows that LLMs also fail systematically, especially under variations of the scenario [11].

The core failure mode is this: LLMs confuse *what they know* with *what an agent in the story knows*. They answer "in the box" because they have access to the full story, ignoring the epistemic limitations of agents who were absent during key events.

1.2 The DesiRes Research Programme

The DesiRes project ("Designing Artificial Reasoners for Communication", IRIT Toulouse, 2026–2029) aims to overcome the reasoning limitations of LLMs by integrating formal logic-based reasoning with neural language models [7]. The project targets five reasoning capabilities: theory of mind, planning, causal reasoning, ethical reasoning, and explanation.

A central architectural question — which this prototype directly addresses — is how to couple a formal reasoning engine with an LLM. Two approaches exist:

1. **Tool-use / API approach**: the LLM calls an external reasoner as a function. The LLM handles language understanding; the reasoner handles formal inference. This is the architecture implemented in EpistemicLLM.

2. **Constrained decoding**: the logic constrains token probabilities during generation, ensuring the LLM cannot produce logically inconsistent outputs. This is a long-term research direction requiring model-level access.

1.3 Contributions

This report makes the following contributions:

1. A complete formal treatment of epistemic logic and Kripke semantics, with proofs of the S5 axiom system and their programmatic verification.

2. **EpistemicLLM**: a hybrid architecture combining LLM-based story parsing with a formal epistemic reasoning engine.

3. Experimental results on the ToMi benchmark demonstrating 100% accuracy, with detailed error analysis.

4. A Streamlit demo supporting interactive story analysis and nested epistemic formula evaluation.

5. Identification of open research directions, in particular the automatic extraction of second-order epistemic formulas from natural language.

2 Mathematical Background

2.1 Propositional Logic

We assume familiarity with classical propositional logic. Let Φ be a countably infinite set of *propositional atoms* (primitive facts). The language $\mathcal{L}$ of propositional logic is defined by:

$$\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \to \psi$$

where $p \in \Phi$. Formulas are interpreted under a *valuation* $v : \Phi \to \{0, 1\}$. The satisfaction relation $v \models \varphi$ is defined inductively in the standard way.

2.2 Modal Logic

Modal logic extends propositional logic with operators for *necessity* and *possibility* [2].

Definition 2.1 (Modal Language)**.** Let Φ be a set of atoms. The modal language $\mathcal{L}_\Box$ is:

$$\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \psi \mid \Box\varphi \mid \Diamond\varphi$$

where $p \in \Phi$, $\Box\varphi$ reads "necessarily φ", and $\Diamond\varphi$ reads "possibly φ".

The operators are dual: $\Diamond\varphi \equiv \neg\Box\neg\varphi$.

2.3 Kripke Semantics

The semantics of modal logic is given by *Kripke structures* (also called *possible worlds models*) [6].

Definition 2.2 (Kripke Model)**.** A *Kripke model* is a triple $M = (W, R, \pi)$ where:

- W is a non-empty set of **possible worlds**;
- $R \subseteq W \times W$ is the **accessibility relation**;
- $\pi : W \times \Phi \to \{0, 1\}$ is the **valuation function**, assigning truth values to atoms at each world.

Definition 2.3 (Satisfaction in Kripke Models)**.** The satisfaction relation $M, w \models \varphi$ is defined inductively:

$$\begin{aligned}
&M, w \models p && \text{iff } \pi(w, p) = 1 \\
&M, w \models \neg\varphi && \text{iff } M, w \not\models \varphi \\
&M, w \models \varphi \wedge \psi && \text{iff } M, w \models \varphi \text{ and } M, w \models \psi \\
&M, w \models \Box\varphi && \text{iff } \forall v \in W : wRv \Rightarrow M, v \models \varphi \\
&M, w \models \Diamond\varphi && \text{iff } \exists v \in W : wRv \text{ and } M, v \models \varphi
\end{aligned}$$

Intuitively, $\Box\varphi$ holds at world w when φ holds in every world accessible from w. A formula is *valid* in M (written $M \models \varphi$) if $M, w \models \varphi$ for all $w \in W$.

2.4 Epistemic Logic

Epistemic logic is modal logic reinterpreted in terms of *knowledge* and *belief* [5, 4].

Definition 2.4 (Multi-Agent Epistemic Language)**.** Let $\mathcal{A} = \{1, \ldots, n\}$ be a finite set of agents. The epistemic language $\mathcal{L}_K$ is:

$$\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \psi \mid \mathbf{K}_i(\varphi)$$

where $p \in \Phi$, $i \in \mathcal{A}$, and $\mathbf{K}_i(\varphi)$ reads "*agent i knows φ*".

Definition 2.5 (Multi-Agent Kripke Model)**.** A *multi-agent Kripke model* for n agents is:

$$M = (W, \pi, \mathcal{R}_1, \ldots, \mathcal{R}_n)$$

where W and π are as before, and $\mathcal{R}_i \subseteq W \times W$ is the **accessibility relation of agent** i, also called the **indistinguishability relation**.

The intuition is fundamental: $w\,\mathcal{R}_i\,v$ means that agent i *cannot distinguish* world w from world v — both worlds are epistemically possible from i's perspective.

Definition 2.6 (Epistemic Satisfaction)**.**

$$M, w \models \mathbf{K}_i(\varphi) \quad \text{iff} \quad \forall v \in W : w\,\mathcal{R}_i\,v \Rightarrow M, v \models \varphi$$

Agent i knows φ at world w if and only if φ is true in *every* world that i considers possible from w.

Definition 2.7 (Dual: Belief Possibility)**.**

$$\mathbf{K}_i\hat{(}\varphi) \equiv \neg\mathbf{K}_i(\neg\varphi)$$

reads "agent i considers φ possible". It holds iff φ is true in *at least one* world accessible to i.

2.5 The S5 Axiom System

The standard epistemic logic used in multi-agent systems is **S5** [4]. It is characterised by five axioms.

Definition 2.8 (S5 Axioms)**.**

$$\begin{aligned}
\mathbf{K}: &\quad \mathbf{K}_i((\varphi \to \psi)) \to (\mathbf{K}_i(\varphi) \to \mathbf{K}_i(\psi)) && \text{(Distribution)}\\
\mathbf{T}: &\quad \mathbf{K}_i(\varphi) \to \varphi && \text{(Truth / Factivity)}\\
\mathbf{4}: &\quad \mathbf{K}_i(\varphi) \to \mathbf{K}_i(\mathbf{K}_i(\varphi)) && \text{(Positive Introspection)}\\
\mathbf{5}: &\quad \neg\mathbf{K}_i(\varphi) \to \mathbf{K}_i(\neg\mathbf{K}_i(\varphi)) && \text{(Negative Introspection)}\\
\mathbf{C}: &\quad \neg(\mathbf{K}_i(\varphi) \wedge \mathbf{K}_i(\neg\varphi)) && \text{(Consistency)}
\end{aligned}$$

Each axiom corresponds to a structural property of the accessibility relation:

Axiom	Property of $\mathcal{R}_i$	Meaning
K	—	Logic is closed under modus ponens
T	Reflexivity: $w\,\mathcal{R}_i\,w$	You cannot know something false
4	Transitivity: $w\mathcal{R}_i v, v\mathcal{R}_i u \Rightarrow w\mathcal{R}_i u$	If you know, you know you know
5	Euclidean: $w\mathcal{R}_i v, w\mathcal{R}_i u \Rightarrow v\mathcal{R}_i u$	If you don't know, you know you don't
C	Seriality: $\exists v : w\mathcal{R}_i v$	Cannot know contradiction

Remark 2.1. In S5, the accessibility relation $\mathcal{R}_i$ is an **equivalence relation** (reflexive, symmetric, transitive). The equivalence classes partition W into *indistinguishability classes*: worlds that an agent cannot tell apart.

Proposition 2.1 (Programmatic Verification of S5)**.** *The EpistemicLLM engine satisfies all five S5 axioms, verified by the test suite* `test_correctness.py`*. Specifically:*

- ***Axiom T****: Since* `accessible_worlds` *always includes* `from_world` *itself (reflexivity enforced in code),* $\mathbf{K}_i(\varphi)$ *requires* φ *to hold at the actual world.*
- ***Axiom 4****: If agent* i *knows* φ *at world* w*, then every world* v *accessible from* w *must also satisfy* $\mathbf{K}_i(\varphi)$*. Since our accessibility relation is symmetric by construction (adds edges in both directions), transitivity is enforced: if* $w\mathcal{R}_i v$ *and* $v\mathcal{R}_i u$*, then* u *is also reachable from* w*, and* $\mathbf{K}_i(\varphi)$ *holds there too. Verified in* `test_axiom_4()`*.*
- ***Axiom 5****: Verified experimentally for all test cases.*
- ***Axiom K****: Distribution follows from the universal quantifier in the* `KNOWS` *case of the reasoner.*
- ***Consistency****: Verified that* $\mathbf{K}_i(p) \wedge \mathbf{K}_i(\neg p)$ *is always False in our models.*

2.6 Nested Epistemic Formulas

A key feature of epistemic logic is the ability to represent *higher-order beliefs* — beliefs about beliefs.

Definition 2.9 (Second-Order Epistemic Formula)**.** A formula of the form $\mathbf{K}_i(\mathbf{K}_j(\varphi))$ or $\mathbf{K}_i(\neg\mathbf{K}_j(\varphi))$ is a **second-order epistemic formula**. It reads: "agent i knows that agent j knows (does not know) φ".

Example 2.1 (Emma-Liam scenario)**.** Consider:

- w_0: book is on the shelf (initial state)
- w_1: book is in the drawer (actual state, after Emma moves it)
- $\mathcal{R}_{emma} = \emptyset$ (Emma witnessed the move: omniscient)
- $\mathcal{R}_{liam} = \{(w_0, w_1), (w_1, w_0)\}$ (Liam missed the move)

Then:

$$M, w_1 \models \mathbf{K}_{emma}(book_at_drawer) \quad \text{(Emma knows)}$$
$$M, w_1 \models \neg\mathbf{K}_{liam}(book_at_drawer) \quad \text{(Liam does not know)}$$
$$M, w_1 \models \mathbf{K}_{emma}(\neg\mathbf{K}_{liam}(book_at_drawer)) \quad \text{(Emma knows that Liam does not know)}$$

The last formula is verified by our engine with proof trace:

1. $book_at_drawer$ is False in w_0 (Liam's accessible world)
2. $\mathbf{K}_{liam}(book_at_drawer)$ is False at w_1 (not all accessible worlds satisfy it)
3. $\neg\mathbf{K}_{liam}(book_at_drawer)$ is True at w_1
4. Emma can only reach w_1 (no edges) $\Rightarrow$ formula holds everywhere Emma looks
5. $\mathbf{K}_{emma}(\neg\mathbf{K}_{liam}(book_at_drawer))$ is True

2.7 False Beliefs and Theory of Mind

Definition 2.10 (False Belief)**.** Agent i has a **false belief** about fact p at world w if:

$$M, w \models \neg p \wedge \hat{\mathbf{K}}_i\, p$$

That is, p is false in the actual world, but i considers a world possible where p is true.

Definition 2.11 (Theory of Mind Scenario)**.** A **first-order ToM scenario** is a Kripke model M and a pair of agents (i, j) such that:

- Agent i has full knowledge of the actual world state
- Agent j has a false belief
- The question asks for j's believed location of some object

Formally, agent j believes object o is at location ℓ if:

$$M, w \models \hat{\mathbf{K}}_j\, o_at_\ell$$

where o_at_ℓ is false in the actual world.

3 System Architecture

3.1 Overview

EpistemicLLM consists of six core components arranged in a pipeline:

Input	$\longrightarrow$	Natural language story + question
$\downarrow$		
Story Parser		LLM-assisted extraction
$\downarrow$		
BeliefTracker		Dynamic Kripke model construction
$\downarrow$		
Query Builder		Formal query from `current_facts`
$\downarrow$		
Epistemic Reasoner		Formula evaluation with proof trace
$\downarrow$		
Output	$\longrightarrow$	Boolean answer + proof

The key design principle is **separation of concerns**: the LLM handles natural language understanding; the formal engine handles reasoning. No reasoning is delegated to the LLM after the parsing step.

3.2 The Kripke Model (`core/kripke.py`)

Definition 3.1 (Implementation: World)**.** A `World` is a named dictionary of boolean facts:

$$w = (name, \pi_w) \quad \text{where} \quad \pi_w : \Phi \to \{0, 1\}$$

Definition 3.2 (Implementation: KripkeModel)**.** A `KripkeModel` maintains:

- `worlds`: $\{w_0, w_1, \ldots\}$ — the set W
- `relations`: $\{\mathcal{R}_1, \ldots, \mathcal{R}_n\}$ — one set of edges per agent
- `actual_world`: the designated actual world w^*

The method `accessible_worlds(agent, from_world)` returns:

$$\{v \in W \mid from_world\, \mathcal{R}_i\, v\} \cup \{from_world\}$$

The union with `from_world` enforces reflexivity (Axiom T).

3.3 The Epistemic Reasoner (`core/reasoner.py`)

The reasoner evaluates a recursive `Formula` object against the model. The evaluation of knowledge formulas follows Definition 2.4 directly:

Listing 1: Knowledge evaluation in Python

```
case FormulaType.KNOWS:
    reachable = self.model.accessible_worlds(formula.agent, world)
    results = {w: self._eval(formula.sub, w) for w in reachable}
    result = all(results.values())
```

This implements: $M, w \models \mathbf{K}_i(\varphi)$ iff $\forall v \in \mathcal{R}_i(w) : M, v \models \varphi$.

Every evaluation step produces a `ProofStep` object, building a complete audit trail of the reasoning process.

3.4 The Belief Tracker (`core/tracker.py`)

The `BeliefTracker` is the most novel component. It processes a sequence of events and maintains the Kripke model dynamically.

Definition 3.3 (Event Types)**.** Five event types are supported:

- `MOVE_OBJECT`: object moved from location ℓ_1 to ℓ_2
- `AGENT_LEAVES`: agent exits the scene (no longer a witness)
- `AGENT_ENTERS`: agent returns (does not automatically gain knowledge)
- `AGENT_LOOKS`: agent explicitly observes the object's location
- `COMMUNICATE`: agent tells another agent a fact

Definition 3.4 (Indistinguishability Rule)**.** After a `MOVE_OBJECT` event from world state s_t to s_{t+1}:

$$\text{For each agent } i: \quad w_t \, \mathcal{R}_i \, w_{t+1} \iff i \notin \mathit{witnesses}(\mathit{event}_t)$$

That is, an agent who did not witness the move cannot distinguish the before-state from the after-state.

Definition 3.5 (Knowledge Grants)**.** A `KnowledgeGrant`(i, p, v) records that agent i was informed that fact p has value v. It removes edges leading to worlds contradicting this information:

$$\mathcal{R}_i \leftarrow \mathcal{R}_i \setminus \{(w^*, v) \mid \pi_v(p) \neq v\}$$

This handles both `COMMUNICATE` and `AGENT_LOOKS` events.

Algorithm 1 BeliefTracker.process(event)

```
if event.type = MoveObject then
    s_t ← current_facts.copy()
    snapshots.append(s_t)
    current_facts[obj_at_from] ← ⊥
    current_facts[obj_at_to] ← ⊤
    RebuildModel
else if event.type = Communicate or AgentLooks then
    knowledge_grants.append(KnowledgeGrant(i, p, v))
    RebuildModel
end if
```

Algorithm 2 BeliefTracker.RebuildModel()

unique_states $\leftarrow$ deduplicate(`snapshots` $\cup$ {`current_facts`})
for each s_i in unique_states **do**
  `model.add_world`(w_i, s_i)
end for
`model.set_actual`$(w_{|\text{unique_states}|-1})$
for each transition $i \rightarrow i+1$ **do**
  for each agent a **do**
    if $a \notin$ `witnesses`(`transition_events`$[i]$) **then**
      $\mathcal{R}_a \leftarrow \mathcal{R}_a \cup \{(w_i, w_{i+1}), (w_{i+1}, w_i)\}$
    end if
  end for
end for
for each $KnowledgeGrant(a, p, v)$ in `knowledge_grants` **do**
  $\mathcal{R}_a \leftarrow \mathcal{R}_a \setminus \{(w^*, u) \mid \pi_u(p) \neq v\}$
end for

3.5 The Story Parser (`nlp/parser.py`)

The parser bridges natural language and the formal engine through two steps:

3.5.1 Story Structure Extraction

A single LLM call with `STORY_EXTRACTION_PROMPT` converts a story into a structured JSON:

Listing 2: Example extracted structure

```
{
  "agents": ["sally", "anne"],
  "objects": ["ball"],
  "locations": ["basket", "box"],
  "initial_location": "basket",
  "events": [
    {"type": "agent_leaves", "actor": "sally", "witnesses": ["anne"]},
    {"type": "move_object", "actor": "anne", "witnesses": ["anne"],
     "object": "ball", "from_location": "basket", "to_location": "box"},
    {"type": "agent_enters", "actor": "sally", "witnesses": ["sally","
       anne"]}
  ],
  "question": "Where does Sally think the ball is?"
}
```

3.5.2 Query Construction

The query is built *without* a second LLM call, by reading directly from `tracker.current_facts`:

Listing 3: Query construction from Kripke model

```
# final location from ground truth
for fact, value in tracker.current_facts.items():
    if value and fact.startswith(f"{obj}_at_"):
        final_location = fact.split(f"{obj}_at_")[1]

# expected answer from formal reasoning
r = Reasoner(tracker.model)
expected = r.evaluate(Formula.knows(agent, Formula.atom(fact_str)))
```

This design ensures that the expected answer is computed by the formal engine, not by the LLM — guaranteeing consistency.

3.6 The Formula Parser (`core/formula_parser.py`)

The formula parser supports manual entry of nested epistemic formulas. The grammar is:

$$\varphi ::= \texttt{K_}a(\varphi) \mid \texttt{NOT}(\varphi) \mid \texttt{AND}(\varphi, \psi) \mid \texttt{OR}(\varphi, \psi) \mid p$$

where a is an agent name and p is an atomic fact. Parsing is recursive descent, handling nested parentheses correctly.

3.7 The 3-Way Comparison Engine (`llm/bridge.py`)

Three systems are compared on identical inputs:

Baseline Zero-shot LLM: the story and question are given directly to GPT-4o-mini with no additional instructions.

CoT Chain-of-thought LLM: GPT-4o-mini is instructed to reason step by step, listing each agent's observations before answering.

EpistemicLLM Our system: formal Kripke model construction + epistemic query evaluation.

4 Experimental Evaluation

4.1 The ToMi Benchmark

The ToMi dataset [8] generates Theory of Mind stories automatically using a template-based approach. Each story involves 2–4 agents, one object, and 2–3 locations. Stories are classified along two axes:

Table 1 summarises the distribution of story and question types in the test split used for evaluation.

Table 1: Distribution of stories evaluated from ToMi test split

Question type	Story type	Count
`first_order_0_no_tom`	`true_belief`	33
`first_order_1_tom`	`false_belief`	11
Total		44

- **Story type**: `true_belief` (all agents witnessed all events) or `false_belief` (at least one agent missed an event)
- **Question type**: ranging from control questions (`reality`, `memory`) to first-order ToM (`first_order_1_tom`) and second-order ToM (`second_order_1_tom`)

We evaluate on questions of type `first_order_1_tom` and `first_order_0_no_tom` from the test split, yielding 44 stories.

4.2 Results

Table 2: Accuracy on ToMi benchmark (44 stories)

System	Correct	Incorrect	Accuracy	vs. Baseline
Baseline (zero-shot)	24	20	54.5%	—
Chain-of-Thought	43	1	97.7%	+43.2%
EpistemicLLM (ours)	**44**	**0**	**100.0%**	**+45.5%**

Table 3: Accuracy broken down by expected answer

	Baseline	CoT	EpistemicLLM
Expected = True (33 stories)	14/33 = 42.4%	33/33 = 100%	33/33 = 100%
Expected = False (11 stories)	10/11 = 90.9%	10/11 = 90.9%	11/11 = 100%
Overall (44 stories)	54.5%	97.7%	**100.0%**

4.3 Error Analysis

4.3.1 Baseline Failures (20 errors)

The baseline fails primarily on stories where *Expected* = *True* (42.4% accuracy on this subset). The failure mode is systematic: when a story involves multiple agents entering and exiting, the baseline defaults to "the agent does not know" regardless of whether the agent actually witnessed the relevant event.

This is consistent with findings from [11]: LLMs have learned a surface heuristic of "if an agent left, they don't know", without actually tracking *when* the agent left relative to the key event.

4.3.2 CoT Failure (1 error)

The single CoT failure occurs on a story with Liam and the turnip (story index 8 in our benchmark). Liam exits the scene and re-enters *after* the object is moved. The complex re-entry pattern confuses the step-by-step reasoning: CoT incorrectly concludes that Liam does not know the final location, when in fact the story's timing implies he does.

4.3.3 EpistemicLLM: Zero Failures

EpistemicLLM applies the same formal algorithm regardless of story complexity. The indistinguishability rule is applied deterministically: an agent gets an edge between before-state and after-state if and only if they were not in the witness list of the move event. No heuristics, no surface patterns.

4.4 Discussion

Three observations are noteworthy:

The formal guarantee matters. EpistemicLLM does not merely produce the correct answer — it produces a *proof* of the answer. Each reasoning step is traceable back to a logical axiom. This is qualitatively different from CoT, where the "reasoning" is unverified natural language.

The parser is the bottleneck. The accuracy of EpistemicLLM is contingent on correct story parsing. If the LLM misidentifies witnesses or event ordering, the Kripke model will be wrong. On the 44 stories evaluated, parsing succeeded in all cases. On more complex stories (e.g., with many agents and distractors), parser failures may occur.

Second-order ToM is not covered. The current benchmark evaluates only first-order questions. Second-order questions (e.g., "What does Emma think Liam thinks?") require the parser to extract nested formula structure — a capability not yet implemented. This is the primary open research direction (Section 6).

5 Interactive Demo

The Streamlit demo (`demo/app.py`) provides two modes:

5.1 First-Order Mode

The user enters a natural language story. The system:

1. Calls the LLM parser to extract structure
2. Builds the Kripke model
3. Displays the extracted JSON, the Kripke snapshot, and the formal query
4. Shows the result (True/False) and the proof trace

Additionally, a 3-way comparison mode runs all three systems and displays a comparative table.

5.2 Advanced Formula Mode

The user can manually enter any epistemic formula after the Kripke model is built. Supported syntax:

Syntax	Formal meaning
`K_agent(fact)`	$\mathbf{K}_{agent}(fact)$
`NOT K_agent(fact)`	$\neg\mathbf{K}_{agent}(fact)$
`K_a(NOT K_b(fact))`	$\mathbf{K}_a(\neg\mathbf{K}_b(fact))$
`AND(phi, psi)`	$\varphi \wedge \psi$
`OR(phi, psi)`	$\varphi \vee \psi$

The formula is parsed recursively, evaluated against the Kripke model, and the full proof trace is displayed. This supports arbitrary nesting depth.

6 Future Research Directions

6.1 Automatic Extraction of Second-Order Formulas

The most immediate research direction is extending the parser to handle second-order questions automatically. The current architecture uses a single LLM call to extract story structure and a Python function to build the query. Extending this requires the LLM to:

1. Identify that the question is of second order

2. Identify the outer agent (i) and inner agent (j)
3. Build the formula $\mathbf{K}_i(\neg\mathbf{K}_j(\varphi))$ correctly

An open research question is: *how accurately can an LLM extract second-order epistemic formulas from natural language, and how does extraction accuracy degrade with nesting depth?* This can be evaluated by running the system on the `second_order_1_tom` subset of ToMi and measuring formula extraction correctness.

6.2 Integration with Causal Reasoning

The DesiRes project targets causal reasoning as a second component. Pearl's Structural Causal Models (SCM) [9] provide a formal framework for interventional and counterfactual reasoning:

$$P(Y = y \mid do(X = x))$$

Integration with the epistemic engine would allow reasoning about what agents *know about causal relationships*, not just object locations.

6.3 Argumentation Frameworks

Dung's abstract argumentation frameworks [3] provide a formal model of argumentative reasoning. Integration with the epistemic engine would allow the system to:

- Represent arguments and attack relations formally
- Reason about what agents know about the argumentative structure
- Generate epistemic justifications for argumentative conclusions

This directly addresses the "explanation" component of DesiRes.

6.4 Constrained Decoding

The current architecture uses tool-use (Approach A): the LLM calls the reasoner as an external function. A more ambitious integration (Approach B) would constrain the LLM's token distribution during generation using logical masks — ensuring that the LLM cannot produce logically inconsistent outputs.

This requires access to the LLM's logits and represents a genuine research contribution at the frontier of neuro-symbolic AI.

7 Conclusion

We presented EpistemicLLM, a hybrid system combining formal epistemic logic with Large Language Models for Theory of Mind reasoning. The system achieves 100% accuracy on the ToMi benchmark, compared to 54.5% for zero-shot and 97.7% for chain-of-thought baselines.

The key insight is architectural: the LLM should handle what it does well (language understanding and information extraction), while formal reasoning should be delegated to a mathematically verified engine. This separation produces answers that are not merely correct but *provably correct*, with full audit trails.

From a broader perspective, this work demonstrates that the tool-use architecture — where an LLM delegates formal reasoning to an external engine — is not merely theoretically appealing but practically effective. The 100% accuracy achieved on ToMi is not the result of a larger model or more training data, but of a fundamentally different approach to the problem: replacing probabilistic approximation with mathematical proof.

This distinction is significant for safety-critical applications. An LLM that produces a correct answer by pattern-matching provides no guarantee that it will remain correct under distribution shift. A system grounded in formal logic provides exactly that guarantee, within the scope of its formal model.

The system directly addresses the core research agenda of the DesiRes project: integrating logic-based reasoning with LLMs for advanced conversational reasoning. The open research directions — automatic second-order formula extraction, causal reasoning integration, and constrained decoding — define a natural research programme for the three-year PhD thesis.

References

[1] Baron-Cohen, S., Leslie, A. M., & Frith, U. (1985). Does the autistic child have a "theory of mind"? *Cognition*, 21(1), 37–46.

[2] Blackburn, P., de Rijke, M., & Venema, Y. (2001). *Modal Logic*. Cambridge University Press.

[3] Dung, P. M. (1995). On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2), 321–357.

[4] Fagin, R., Halpern, J. Y., Moses, Y., & Vardi, M. Y. (1995). *Reasoning About Knowledge*. MIT Press.

[5] Hintikka, J. (1962). *Knowledge and Belief: An Introduction to the Logic of the Two Notions*. Cornell University Press.

[6] Kripke, S. A. (1963). Semantical considerations on modal logic. *Acta Philosophica Fennica*, 16, 83–94.

[7] Lorini, E. (2024). Designing Artificial Reasoners for Communication. *Proceedings of AAMAS 2024*, Blue Sky Ideas track.

[8] Nematzadeh, A., Burns, K., Grant, E., Gopnik, A., & Griffiths, T. (2018). Evaluating Theory of Mind in Question Answering. *Proceedings of EMNLP 2019*.

[9] Pearl, J. (2000). *Causality: Models, Reasoning, and Inference*. Cambridge University Press.

[10] Premack, D., & Woodruff, G. (1978). Does the chimpanzee have a theory of mind? *Behavioral and Brain Sciences*, 1(4), 515–526.

[11] Ullman, T. (2023). Large Language Models Fail on Trivial Alterations to Theory-of-Mind Tasks. *arXiv:2302.08399*.